

SciSearch: Indicizzazione e testing su ArXiv e PubMed

Piergiorgio Fornaro, Valeria De Falco, Fiamma
Sbrega, Andrea Leopardi

January 20, 2026

Indice

1. Architettura	3
1.1. Scrapers	3
1.1.1. arxiv_scraper	3
1.1.2. pubmed_scraper	4
1.2. Extraction	5
1.2.1. Riconoscimento del Formato	6
1.2.2. Gestione Documenti ArXiv	6
1.2.3. Gestione Documenti PubMed	7
1.2.4. Context Mapping e Arricchimento Semantico	7
1.3. Indexing	8
1.3.1. Scelta Tecnologica: Elasticsearch	8
1.3.2. Strategia di Indicizzazione	8
1.3.3. Workflow di Caricamento	10
2. Valutazione Sperimentale	12
2.1. Analisi quantitativa	12
2.1.1. Risultati	13
2.2. Analisi Qualitativa	14
2.2.1. Risultati	15
2.3. Conclusioni	17

1. Architettura

1.1. Scrapers

Il modulo di scraping è responsabile dell'acquisizione automatizzata dei documenti. Sono stati sviluppati due script dedicati per interrogare le API specifiche delle sorgenti selezionate:

- **arxiv_scraper.py** per l'ambito tecnico-informatico.
- **pubmed_scraper.py** per l'ambito medico-scientifico.

Entrambi gli script implementano meccanismi di **resumability** (controllo dei file già scaricati) e **rate limiting** (attese tra le richieste) per rispettare le policy dei server.

1.1.1. arxiv_scraper

Questo script gestisce il recupero dei paper da **ArXiv**. A differenza di PubMed, ArXiv non fornisce XML strutturato nativamente; tuttavia, per i paper recenti, ArXiv offre una versione HTML sperimentale generata tramite `arxiv`. Lo script mira a scaricare questa versione HTML per facilitare la successiva estrazione di tabelle e figure.

Il funzionamento si basa sulla libreria `arxiv` e `requests` e si basa su tre passi principali:

1. **Configurazione Client e Ricerca:** Viene istanziato un `arxiv.Client` configurato con parametri di ritardo e riprova automatici per gestire la stabilità della connessione. La ricerca viene effettuata ordinando per rilevanza. Viene passata la query richiesta dall'utente.

```
client = arxiv.Client(
    page_size=100,
    delay_seconds=3.0, # Anti rate-limit nativo della libreria
    num_retries=3
)
search = arxiv.Search(query=query, sort_by=arxiv.SortCriterion.Relevance)
```

2. **Verifica Disponibilità HTML:** ArXiv permette di visualizzare ed ottenere l'HTML di un paper conoscendone l'id ed effettuando una richiesta tramite `https://arxiv.org/html/{paper_id}`. Lo script effettua una richiesta HTTP preliminare. Se ArXiv risponde con un redirect verso `/abs/{id}`, significa che la versione HTML non è disponibile (comune per i paper più vecchi); in tal caso, il documento viene saltato.

```
html_url = f"[https://arxiv.org/html/](https://arxiv.org/html/){paper_id}"
response = requests.get(html_url, headers=headers, timeout=15)

# Verifica redirect (assenza di HTML)
if "abs/" in response.url:
    print(f"-> HTML not found. Skipping.")
    continue
```

3. **Salvataggio e Rate Limiting:** Se il controllo sull'url ha esito positivo, il contenuto HTML viene salvato su disco. Contestualmente, vengono estratti i metadati forniti dall'oggetto `result` della libreria `arxiv` (titolo, abstract, data, autori) e salvati in JSON. Un'attesa esplicita di 2 secondi viene rispettata tra i download per evitare il ban dell'indirizzo IP.

```
# Save Metadata
metadata = {
    "id": paper_id,
    "title": result.title,
    "published": result.published.isoformat(),
    "pdf_url": result.pdf_url,
    # ...
}
```

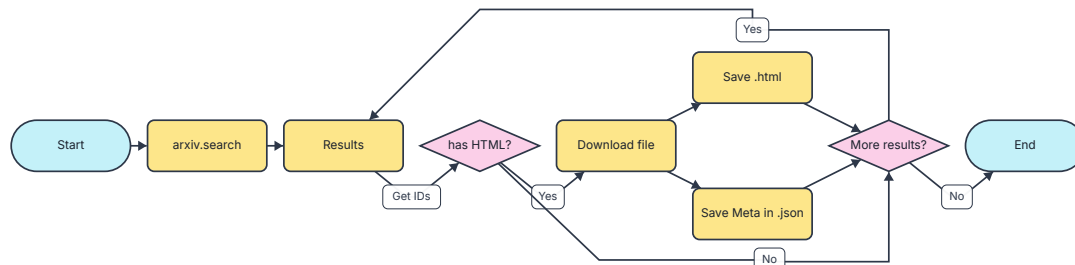
```

}

# Delay (ArXiv è sensibile ai download di massa)
time.sleep(2)

```

Il flusso logico è riassunto nel seguente diagramma:



1.1.2. pubmed_scraper

Questo script gestisce il download massivo da **PubMed Central (PMC)** utilizzando la libreria `Bio.Entrez`. La scelta del formato XML è dettata dalla necessità di ottenere dati strutturati e bypassare le restrizioni di accesso spesso imposte sulle pagine HTML classiche.

Il flusso di esecuzione si articola in tre fasi:

1. **Configurazione e Ricerca:** Si imposta un indirizzo email universitario, così da risultare studente ed aggirare blocchi anti-bot di sicurezza. Dopodiché si compone la query, formata dalla parola chiave cercata ed il filtro `AND open access[filter]`, così da ricercare solamente articoli disponibili gratuitamente. Si ordina poi per rilevanza.

```

Entrez.email = "student@university.edu"
# Headers per simulare un browser reale
HEADERS = { "User-Agent": "Mozilla/5.0 ...", ... }

full_query = f"{query} AND open access[filter]"
try:
    handle = Entrez.esearch(db="pmc", term=full_query, retmax=max_results,
sort="relevance")
    record = Entrez.read(handle)
    handle.close()
    // altro codice...

```

2. **Deduplica e Validazione:** Per ogni elemento trovato legge l'ID e, se non presente, aggiunge «PMC» antecedentemente alla stringa numerica. Ciò è necessario per poter poi scaricare il file .xml dal loro sito. Si effettua poi un controllo: se il file .xml è già stato scaricato in precedenza lo script ignora tale file e continua, evitando così di creare duplicati.

```

pmc_id = f"PMC{pmc_id_raw}" if not pmc_id_raw.startswith("PMC") else pmc_id_raw

# Check if already exists (XML or HTML)
filename = f"{pmc_id}.xml"
filepath = os.path.join(DATA_DIR_PM, filename)
if os.path.exists(filepath):
    print(f" -> Already exists. Skipping.")
    count += 1
    continue

```

3. **Download e Parsing Metadati:** Il contenuto XML viene scaricato tramite Entrez.efetch. Successivamente, BeautifulSoup effettua il parsing dell'albero XML per estrarre metadati granulari (Titolo; Autore/i, Abstract e Data). È stata implementata una logica robusta per la normalizzazione della data, che tenta di recuperare in ordine di priorità la data di pubblicazione elettronica (epub), cartacea (ppub) o di rilascio (pmc-release), gestendo anche la conversione dei mesi da formato testuale a numerico. Poiché PubMed impone un limite di 3 richieste al secondo, tra un documento e l'altro si inserisce uno sleep di 0.34s.

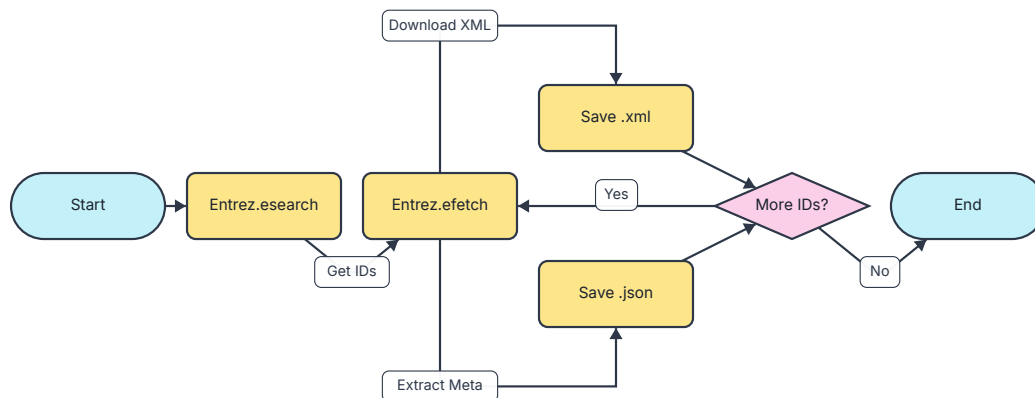
```
# Download XML
handle = Entrez.efetch(db="pmc", id=pmc_id, rettype="full", retmode="xml")

# Estrazione robusta della data
pub_date = soup.find("pub-date", {"pub-type": "epub"}) or \
            soup.find("pub-date", {"pub-type": "ppub"}) or ...

metadata = {
    "id": pmc_id,
    "title": title,
    "authors": authors,
    "date": date_str, # Formato ISO 8601 (YYYY-MM-DD)
    "source": "pubmed"
    # ...
}

# Rate limiting (0.34s come da linee guida NCBI)
time.sleep(0.34)
```

Il flusso logico è riassunto nel seguente diagramma:



1.2. Extraction

Il modulo di estrazione ha come obiettivo trasformare documenti non strutturati o semi-strutturati (HTML/XML) in oggetti Python strutturati e pronti per l'indicizzazione.

Le responsabilità principali del modulo sono:

- Caricamento e parsing del DOM (Document Object Model).
- Identificazione della fonte (ArXiv o PubMed).
- Estrazione granulare di: testo completo, tabelle (con didascalie), figure (con URL e didascalie) e metadati.

- Analisi del contesto semantico (mapping tra oggetti e paragrafi che li citano).

1.2.1. Riconoscimento del Formato

Il metodo `process_file` funge da unico punto di ingresso per l'elaborazione di un singolo documento. La funzione analizza le caratteristiche del file in input e delega l'estrazione effettiva al metodo specializzato corretto. Il flusso di esecuzione si articola in tre passaggi:

1. **Caricamento e Inizializzazione del Parser:** Il file viene aperto in modalità lettura (`utf-8`) e il contenuto viene passato alla libreria `BeautifulSoup` . La scelta del parser sottostante è dinamica: viene utilizzato il parser `xml` se l'estensione del file è `.xml` , altrimenti si opta per `html.parser` .
2. **Derivazione dell'Identificativo:** Il `paper_id` viene estratto direttamente dal nome del file, rimuovendo le estensioni, per garantire coerenza tra il file system e l'ID logico del documento.
3. **Routing della Strategia:** Il sistema decide quale logica di estrazione applicare (`_process_pubmed` o `_process_arxiv`) basandosi su due criteri:
 - La presenza del prefisso «PMC» nel `paper_id` (tipico di PubMed Central).
 - L'estensione del file originale.

Questo approccio garantisce che, indipendentemente dalla fonte, l'output finale sia sempre un dizionario strutturato uniforme.

```
def process_file(self, filepath):
    """
    Main method to process a single HTML file.
    """
    with open(filepath, 'r', encoding='utf-8') as f:
        content = f.read()

    # Selezione dinamica del parser in base all'estensione
    if filepath.endswith('.xml'):
        soup = BeautifulSoup(content, 'xml')
    else:
        soup = BeautifulSoup(content, 'html.parser')

    # Normalizzazione ID
    paper_id = os.path.basename(filepath).replace('.html', '').replace('.xml', '')

    # Dispatching della strategia corretta
    if paper_id.startswith("PMC") or filepath.endswith('.xml'):
        return self._process_pubmed(soup, paper_id)
    else:
        return self._process_arxiv(soup, paper_id)
```

1.2.2. Gestione Documenti ArXiv

La funzione `_process_arxiv` gestisce il parsing del DOM HTML generato dalla conversione automatica di sorgenti LaTeX (LaTeXML). Le operazioni principali includono:

1. **ID e Deduplicazione:** Il `paper_id` viene normalizzato rimuovendo il suffisso di versione (es. `v1` , `v2`) per garantire che diverse revisioni dello stesso paper mappino sullo stesso oggetto logico nel database.
2. **Estrazione Metadati:** Titolo e Abstract vengono estratti analizzando classi CSS specifiche (`ltx_abstract` , `title`), rimuovendo intestazioni ridondanti come la stringa «Abstract».
3. **Testo Completo:** Viene estratto prioritariamente dal nodo `article.ltx_document` , oppure dal `body` standard.

4. **Tabelle:** Vengono individuate tramite la classe `ltx_table`. Per ogni tabella, viene generato un ID univoco composito (`paper_id` + `table_id`) e viene estratto sia il contenuto testuale (per la ricerca) sia l'HTML raw (per la visualizzazione).
5. **Figure e Deduplicazione:** Le figure vengono estratte cercando i tag `<figure>`. Per risolvere il problema delle figure duplicate (comune nell'HTML di ArXiv dove la stessa immagine può apparire in più formati), il sistema implementa un meccanismo di **deduplicazione basato sull'URL**: gli URL delle immagini vengono normalizzati e tracciati in un `set`; le occorrenze successive dello stesso URL vengono scartate. Inoltre, gli URL relativi vengono convertiti in assoluti combinandoli con il `base_paper_html` versionato.

```
# Deduplicazione immagini
if img_url:
    clean_url_check = img_url.split('?')[0]
    if clean_url_check in seen_img_urls:
        continue # Skip duplicates
    seen_img_urls.add(clean_url_check)
```

1.2.3. Gestione Documenti PubMed

La funzione `_process_pubmed` è progettata per navigare l'albero semantico XML.

1. **ID:** Il sistema ispeziona i tag `<article-id>` per trovare il PMCID o il DOI, usandolo come identificativo primario.
2. **Conversione Tabelle:** Le tabelle XML (`<table-wrap>`) subiscono un processo di trasformazione. Oltre all'estrazione del testo (`body`), il metodo `xml_table_to_html` converte la struttura XML (es. `<row>`, `<entry>`) in HTML standard (`<tr>`, `<td>`) per renderle visualizzabili nel frontend.
3. **Figure:** Vengono estratti i tag `<fig>` e i relativi link `<graphic>`. L'URL dell'immagine viene ricostruito puntando ai server di NCBI (che ospitano le immagini di PubMed), gestendo le estensioni dei file (`.jpg` come default).

```
img_url = f"https://www.ncbi.nlm.nih.gov/pmc/articles/PMC{clean_pmc_id}/bin/{base_href}"
```

1.2.4. Context Mapping e Arricchimento Semantico

Una volta estratti gli oggetti strutturali (tabelle e figure), il metodo `_post_process_context` arricchisce ciascun elemento associandogli i paragrafi del testo che lo citano o lo descrivono. Questa fase è cruciale per dare «contesto» ai dati e viene fatta chiamando il metodo `_fill_context`, che implementa una strategia ibrida per trovare le relazioni:

1. **Menzioni Esplicite:**
 - Per l'XML (PubMed), vengono cercati i tag `<xref ref-type="fig" rid="...">` che puntano esplicitamente all'ID dell'oggetto.
 - Per l'HTML (ArXiv), vengono analizzati i tag `<a>` cercando corrispondenze tra l'attributo `href` e l'ID della figura/tabella.
2. **Contesto Semantico:** Quando non esistono link espliciti, il sistema utilizza un approccio basato sulle **keyword**.
 - Viene estratto un set di parole chiave dalla didascalia dell'oggetto (rimuovendo stop-word e punteggiatura).
 - Per ogni paragrafo del testo, viene calcolata l'intersezione tra le sue parole chiave e quelle della didascalia.
 - Se l'intersezione supera una soglia definita (es. `ge2` keyword comuni), il paragrafo viene considerato «semanticamente rilevante» e aggiunto alla lista `context_paragraphs`.

```
# Semantic Matching Logic
intersection = keywords.intersection(p_keywords)
if len(intersection) >= 2:
    context_paragraphs.append(p_text)
```

Il risultato finale di questo processo è un dizionario unificato che contiene non solo i dati grezzi, ma una rete di collegamenti semantici tra il testo narrativo e gli oggetti strutturati, pronto per essere indicizzato da Elasticsearch.

1.3. Indexing

Il modulo di indicizzazione trasforma i dati grezzi in strutture ricercabili, garantendone la persistenza e l'organizzazione. A livello implementativo, è stato suddiviso in due componenti distinti:

- **index_manager.py** : definisce i mappings e gestisce l'interazione diretta con Elasticsearch.
- **indexing.py** : coordina le operazioni, gestendo il flusso di caricamento, la verifica dei duplicati e l'unione dei metadati.

1.3.1. Scelta Tecnologica: Elasticsearch

La scelta del motore è ricaduta su **Elasticsearch**. Si tratta di un motore di ricerca e analisi distribuito, *document-oriented* e basato su Apache Lucene. La decisione di utilizzare un motore NoSQL orientato ai documenti, piuttosto che un tradizionale RDBMS, è stata guidata da tre fattori critici:

1. **Ricerca Full-Text e Rilevanza**: A differenza di una query SQL standard, il progetto richiede di ordinare i risultati per rilevanza semantica. Elasticsearch utilizza algoritmi avanzati (come il *BM25*) per calcolare uno *score* di pertinenza, permettendo di premiare, ad esempio, un articolo che menziona una keyword nel titolo o nella caption di una tabella rispetto a uno che la cita solo marginalmente.
2. **Gestione di Oggetti Eterogenei**: Le tabelle e le figure scientifiche sono entità complesse, composte da dati strutturati e testo libero. La natura *schema-free* di Elasticsearch ci ha permesso di modellare questi oggetti preservandone la gerarchia senza le limitazioni rigide delle tabelle relazionali.
3. **Scalabilità**: Dovendo gestire un corpus potenzialmente vasto (ArXiv e PubMed), la capacità nativa di Elasticsearch di scalare orizzontalmente garantisce tempi di risposta nell'ordine dei millisecondi, essenziali per una buona user experience.

1.3.2. Strategia di Indicizzazione

Per soddisfare il requisito progettuale che richiede di trattare tabelle e figure come «oggetti di prima classe», è stata adottata una strategia **multi-indice**, orchestrata dalla classe `IndexManager`. Questa architettura separa logicamente le entità in tre indici distinti (`articles`, `tables`, `figures`), consentendo ricerche granulari (ad esempio, focalizzate esclusivamente sul contenuto tabellare) e ottimizzando le performance di interrogazione.

Di seguito vengono descritti gli schemi (mappings) definiti per ciascun indice.

1. Indice `articles` : Contiene il testo e i metadati dell'articolo.
 - I campi `title`, `abstract` e `full_text` sono analizzati (`text`) per supportare ricerche *full-text*.
 - I campi `authors`, `source` e `date` sono mappati rispettivamente come `keyword` e `date` per permettere filtri esatti e aggregazioni temporali precise.

```
"articles": {
  "mappings": {
    "properties": {
      "title": {"type": "text"},
      "authors": {"type": "keyword"},
```



```

        "date": {"type": "date"},
        "abstract": {"type": "text"},
        "full_text": {"type": "text"},
        "source": {"type": "keyword"} // 'arxiv' or 'pubmed'
    }
}

```

- Indice `tables` : ogni tabella viene indicizzata come un documento autonomo, mantenendo un collegamento logico all'articolo genitore tramite il campo `paper_id`.
 - Campi critici come `caption`, `body` (contenuto delle celle) e `mentions` (paragrafi citanti) sono indicizzati come testo per massimizzare la reperibilità semantica.

```

"tables": {
  "mappings": {
    "properties": {
      "paper_id": {"type": "keyword"},
      "table_id": {"type": "keyword"},
      "caption": {"type": "text"},
      "body": {"type": "text"},
      "mentions": {"type": "text"},
      "context_paragraphs": {"type": "text"},
      "source": {"type": "keyword"}
    }
  }
}

```

- Indice `figures` : l'indice per le figure segue una logica speculare a quella delle tabelle, includendo campi specifici come l' `url` della risorsa grafica.

La classe `IndexManager` espone due metodi:

- `create_indices()`** : Invocata dal modulo `indexer.py`, questa funzione è responsabile del *bootstrap* del database. Itera sulla configurazione predefinita degli indici (`articles`, `tables`, `figures`) e, per ciascuno, verifica l'esistenza tramite `self.es.indices.exists`. In caso negativo, procede alla creazione inviando i mapping specifici.
- `index_data()`** : Questa funzione riceve in input un dizionario unificato (dal modulo `indexing.py`) che rappresenta un singolo articolo completo di tutte le componenti estratte e ne orchestra lo smistamento nei rispettivi indici attraverso tre fasi logiche:

- Estrazione e Normalizzazione**: Vengono estratti i metadati principali (`title`, `authors`, `abstract`, `full_text`). Il `paper_id` funge da chiave primaria (`_id`) per garantire l'unicità. La data (`raw_date`) viene normalizzata recuperando il valore più affidabile tra le fonti disponibili (es. `published` da ArXiv), gestendo esplicitamente i casi di data mancante per prevenire errori in Elasticsearch.

```

raw_date = data.get("date") or data.get("published")

article_source = {
    "paper_id": data["paper_id"],
    // ... altri campi ...
    "date": raw_date,
    "source": data.get("source", "arxiv")
}

article_doc = {
    "_index": "articles",

```

```

        "_id": data["paper_id"],
        "_source": article_source
    }

    # Gestione data vuota
    if data.get("date") and str(data["date"]).strip():
        article_source["date"] = data["date"]

```

2. **Creazione Documenti Derivati:** Per ogni tabella e figura estratta, viene generato un documento separato destinato ai rispettivi indici. Anche in questo caso si utilizzano ID univoci e si includono campi semantici (`caption` , `body` , `context_paragraphs` , `mentions`) e riferimenti alle risorse (`url` per le immagini).

```

for tbl in data.get("tables", []):
    table_doc = {
        "_index": "tables",
        "_id": tbl["table_id"],
        "_source": {
            "paper_id": data["paper_id"],
            "table_id": tbl["table_id"],
            "caption": tbl["caption"],
            // ... altri campi ...
            "context_paragraphs": tbl["context_paragraphs"],
            "source": data.get("source", "arxiv")
        }
    }

```

3. **Bulk Indexing:** Per massimizzare l'efficienza di I/O, viene utilizzata la **Bulk API** di Elasticsearch. Tutte le entità relative a un singolo paper (articolo + N tabelle + M figure) vengono aggregate in una lista di «azioni» e inviate al server in un'unica richiesta HTTP. Questo approccio riduce drasticamente l'overhead di rete e la latenza complessiva.

```

if actions:
    helpers.bulk(self.es, actions)

```

1.3.3. Workflow di Caricamento

Lo script `indexing.py` si occupa dell'inserimento effettivo dei dati iterando sulle directory contenenti i file scaricati. Il flusso operativo si articola nei seguenti passaggi:

1. **Inizializzazione:** All'avvio, viene istanziato `IndexManager` per verificare e garantire l'esistenza degli indici e dei relativi mapping.
2. **Deduplica Preventiva:** Prima di processare un file, viene effettuata una query di controllo su Elasticsearch per verificare la presenza del `paper_id`. Questo meccanismo previene la duplicazione dei documenti e consente la ripresa dello script in caso di interruzioni.

```

# Check if already indexed
if indexer.es.exists(index="articles", id=paper_id):
    print(f" -> Article {paper_id} already indexed. Skipping.")
    continue

```

3. **Data Merging e Arricchimento:** I dati estratti dal parsing dell'HTML vengono fusi con i metadati ufficiali (contenuti nei file `_meta.json` generati in fase di scraping). Questo passaggio è fondamentale per garantire l'autorevolezza delle informazioni (titolo, autori, data), che sovrascrivono eventuali imprecisioni derivanti dal parsing.

```

if os.path.exists(meta_filepath):
    with open(meta_filepath, 'r', encoding='utf-8') as f:

```

```

meta = json.load(f)
# Priorità ai metadati ufficiali
data['title'] = meta.get('title', data.get('title'))
data['authors'] = meta.get('authors', [])
data['date'] = meta.get('published', '')
data['source'] = meta.get('source', data.get('source'))

```

4. **Finalizzazione:** Il dizionario arricchito viene passato all' `IndexManager` per l'indicizzazione effettiva.

```

try:
    indexer.index_data(data)
    print(f" -> Successfully indexed {paper_id} ({data['source']}")
except Exception as e:
    print(f"Failed to index {paper_id}: {e}")

```

2. Valutazione Sperimentale

L'obiettivo di questa fase sperimentale è validare l'efficacia e l'efficienza della soluzione di Information Retrieval implementata. La valutazione è stata condotta lungo due direttrici principali:

- **Analisi Quantitativa** (Performance): Misurazione della latenza di risposta del sistema e del volume di dati recuperati al variare della tipologia di query e dell'indice interrogato.
- **Analisi Qualitativa** (Rilevanza): Verifica della pertinenza dei risultati restituiti, con particolare attenzione alla capacità del sistema di recuperare informazioni non testuali (tabelle e figure) che sarebbero andate perse in una ricerca tradizionale.

Per garantire la riproducibilità degli esperimenti, è stata sviluppata una pipeline di testing automatizzata in Python (`quantitative_test.py`, `qualitative_test.py`) che interroga le API di Elasticsearch, raccoglie le metriche di esecuzione e genera report strutturati in formato CSV.

La configurazione degli indici utilizzata durante i test prevede l'applicazione di pesi specifici (boosting) per privilegiare i campi più rilevanti, come segue:

```
- "articles": ["title^3", "abstract^2", "full_text"],  
- "tables": ["caption^3", "body", "paper_title^1", "context_paragraphs", "mentions"],  
- "figures": ["caption^3", "mentions", "paper_title^1", "context_paragraphs"]
```

Per la misurazione della latenza, le query sono state limitate al recupero dei primi 1.000 documenti, simulando uno scenario realistico di Top-k retrieval per evitare che i tempi di trasferimento di rete di grandi volumi JSON falsassero la misurazione della velocità del motore di ricerca.

2.1. Analisi quantitativa

Per valutare il sistema sotto diversi profili di carico, è stato definito un set di 10 query eterogenee, suddivise in 4 categorie logiche. Queste query spaziano da semplici filtri su metadati a complesse operazioni booleane che coinvolgono il full-text.

1. Ricerca Booleana

Descrizione: Query complesse con operatori logici (AND, OR, NOT) su più termini.

Obiettivo: Stress-test del motore di scoring e gestione intersezioni.

Queries: speech AND text, food OR risk, NOT foods

2. Termine Specifico

Descrizione: Ricerca di frasi esatte o acronimi tecnici (Phrase Match).

Obiettivo: Valutare precisione su concetti multi-parola.

Queries: deep learning, LSTM, emotion recognition

3. Indice Specifico

Descrizione: Ricerca mirata su indici non standard (Tabelle/Figure).

Obiettivo: Dimostrare il valore aggiunto dell'architettura multi-indice nel trovare dati strutturati o visuali.

Queries: confidence interval (Tables), plot (Figures)

4. Campo Specifico

Descrizione: Ricerca limitata a un singolo campo (Titolo/Didascalia).
Obiettivo: Misurare la massima velocità teorica del sistema quando il campo di ricerca è ristretto..

Queries: transformer (Title), error rate (Caption)

2.1.1. Risultati

La tabella seguente offre una sintesi dettagliata delle prestazioni del sistema per ciascuno degli scenari definiti. I dati sono organizzati per evidenziare due aspetti critici:

- 1. **Volume (Total Hits):** Il numero totale di documenti rilevanti identificati, indicatore della copertura del sistema per quella specifica query.
- 2. **Latenza:** Il tempo di risposta è ripartito sui tre indici target (**Articles, Tables, Figures**) per isolare il costo computazionale specifico di ogni struttura dati, confrontandolo poi con il tempo totale di esecuzione (**Total Latency**) percepito dall'utente.

QUERY	TYPE	TOTAL HITS	LAT ART. (ms)	LAT TAB. (ms)	LAT FIG. (ms)	TOTAL LATENCY (ms)
FIELD: transformer (title)	Specific Field	74	31	0	0	34,1
FIELD: error rate (caption)	Specific Field	701	0	19	190	216,19
IDX: confidence interval (tables)	Specific Index	1.034	0	399	0	403,56
IDX: plot (figures)	Specific Index	2.061	0	0	247	250,85
TERM: emotion recognition	Specific Term	2.422	269	105	259	642,09
TERM: LSTM	Specific Term	2.656	314	80	257	659,86
TERM: deep learning	Specific Term	5.049	561	155	405	1.131,49
BOOL: food OR risk	Boolean	5.652	594	501	782	1.888,04
BOOL: speech AND text	Boolean	14.118	573	400	552	1.538,09
BOOL: NOT foods	Boolean	53.398	309	63	158	539,47

Tabella 1: Dettaglio performance per scenario e tipologia di indice.

L'analisi dei tempi di risposta evidenzia una correlazione diretta e significativa tra la complessità strutturale della query e la latenza del sistema. Nella **Tabella 1** sono riportati i valori numerici della latenza media calcolata a partire dai risultati generali. Nella **Figura 1** si osserva facilmente una netta dicotomia prestazionale.

SCENARIO	LATENZA
Boolean	1.321,87
Specific Field	125,15
Specific Index	327,21
Specific Term	811,15
Totale	730,37

Tabella 2: Dettaglio numerico

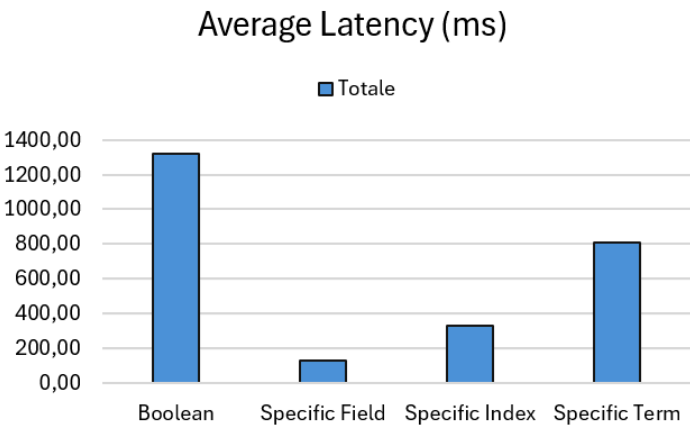


Figura 1: Analisi della latenza media.

Le query di tipo «**Specific Field**» rappresentano lo scenario più performante, con una latenza media di soli **125 ms**. Questo risultato conferma l'efficienza degli indici invertiti di Elasticsearch quando l'interrogazione è puntuale e confinata a singoli campi (es. Titolo), minimizzando le operazioni di I/O e scoring.

Le **query booleane** registrano la latenza più elevata (1.321 ms), mostrando un incremento di un ordine di grandezza rispetto alle query semplici. Tale overhead computazionale è intrinseco alla natura distribuita della ricerca implementata ed è attribuibile alla necessità del motore di:

1. Eseguire la ricerca su tre indici paralleli (Articles, Tables, Figures).
2. Calcolare in tempo reale l'intersezione o l'unione di posting lists massive per termini ad alta frequenza (es. «food» o «risk»).
3. Aggregare i risultati parziali e calcolare il ranking finale basandosi su un algoritmo BM25 che deve normalizzare punteggi provenienti da campi con pesi differenti.

Approfondendo il comportamento del sistema per una query complessa la **Figura 2** scompone la latenza sui tre indici principali.

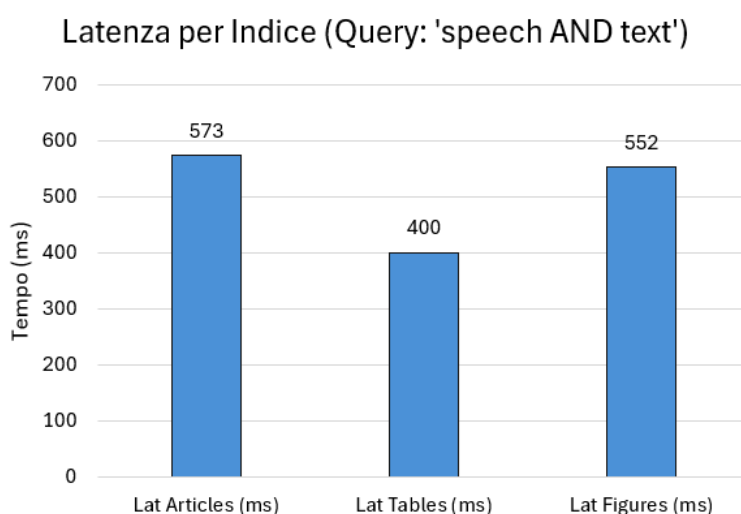


Figura 2: Tempi di elaborazione per ogni indice

Si osserva che l'indice Articles richiede il tempo maggiore (573 ms), seguito a breve distanza dall'indice Figures (552 ms), mentre l'indice Tables risulta il più rapido (400 ms), beneficiando probabilmente di una struttura testuale più schematica e meno discorsiva, che facilita le operazioni di matching.

Mentre la latenza elevata sugli Articoli è attesa, data la dimensione del campo `full_text`, il tempo significativo impiegato per le Figure è un dato notevole: suggerisce che le didascalie (**caption**) e i paragrafi di contesto delle immagini possiedono un'alta densità semantica («information density»), richiedendo al motore di scoring un onere computazionale paragonabile a quello del testo libero.

2.2. Analisi Qualitativa

L'analisi qualitativa ha l'obiettivo di valutare la pertinenza semantica dei risultati restituiti dal sistema. A tale scopo, è stato definito un set di query rappresentative, suddivise per indice target e tipologia di utente (esplorativo vs. tecnico).

Il motore di ricerca implementato supporta nativamente due livelli di specificità grazie all'utilizzo della `query_string` di Elasticsearch:

1. **Ricerca Booleana (Default):** Interpreta gli spazi come operatori `AND` impliciti o accetta operatori espliciti (`OR` , `NOT`). Ideale per ricerche generiche in full-text.
2. **Ricerca Frasale (Phrase Match):** Riconosce l'uso delle virgolette (es. "query") per cercare sequenze esatte di termini. Ideale per definizioni tecniche o entità specifiche.

Di seguito sono descritti gli scenari di test adottati:

1. Ricerca Booleana (Articles)

Descrizione: Query complessa con operatore AND su indice articoli.

Obiettivo: Simula una ricerca medica generica: l'ordine delle parole non è vincolante, ma è richiesta la copresenza di entrambi i concetti nel documento.

Queries: cardiovascular risk AND ultra-processed foods

2. Termine Specifico (Articles)

Descrizione: Ricerca di frase esatta (Phrase Match) per concetti tecnici.

Obiettivo: Valuta la precisione su concetti multi-parola, escludendo contesti in cui i termini appaiono disgiunti.

Queries: "attention mechanism"

3. Dati Strutturati (Tables)

Descrizione: Ricerca mirata all'indice delle tabelle.

Obiettivo: Dimostra il valore aggiunto dell'architettura multi-indice nel reperire dati statistici spesso persi nei PDF tradizionali.

Queries: "confidence interval"

4. Asset Visivi (Figures)

Descrizione: Ricerca limitata alle didascalie dei grafici.

Obiettivo: Dimostra la capacità di trovare visualizzazioni specifiche sfruttando il boosting sulla didascalia.

Queries: "learning curve"

La valutazione è stata condotta mediante una **Top-k Analysis** con $k = 5$, sottoponendo i primi cinque risultati di ogni query a revisione manuale per determinarne la rilevanza binaria (Sì/No).

2.2.1. Risultati

Nel primo scenario, volto a testare la capacità del sistema di correlare concetti medici, tutti i primi 5 risultati si sono rivelati pertinenti (Precision@5 = 100%).

Query: 'cardiovascular risk AND ultra-processed foods' | Indice: articles | Tipo: BOOL

#1 [Score: 48.50] ID: PMC6538975

TITOLO: Ultra-processed food intake and risk of cardiovascular disease: prospective cohort study (NutriNet-Santé)

- Rilevante? (y/n): y

#2 [Score: 45.58] ID: PMC8532572

TITOLO: Ultra-processed Foods, Weight Gain, and Co-morbidity Risk\

- Rilevante? (y/n): y

#3 [Score: 44.46] ID: PMC10623817

TITOLO: Associations of ultra-processed food consumption, circulating protein biomarkers, and risk of cardiovascular disease\

- Rilevante? (y/n): y

#4 [Score: 43.97] ID: PMC8854325

TITOLO: A score appraising Paleolithic diet and the risk of cardiovascular disease in a Mediterranean prospective cohort

- Rilevante? (y/n): y

#5 [Score: 43.38] ID: PMC11494205

TITOLO: Association of ultra-processed food consumption with cardiovascular risk factors among patients with type-2 diabetes mellitus

- Rilevante? (y/n): y

Un caso di particolare interesse è rappresentato dal **risultato 2** («Ultra-processed Foods, Weight Gain, and Co-morbidity Risk»). Sebbene il termine «cardiovascular» non appaia esplicitamente nel titolo, il documento è stato correttamente recuperato con uno score elevato (45.58). L'analisi del contenuto rivela che il termine è presente nel corpo del testo e nelle tabelle interne (**Figura 3**), dove viene discussa la correlazione tra obesità e rischio cardiovascolare.

ID: PMC8532572_Tab3

Narrative reviews, systematic reviews, and meta-analyses summarizing recent evidence evaluating the association between ultra-processed food (UPF) intake and health outcomes

Reference	Type of review	Articles reviewed and study design	Outcome	Key findings
Askari et al. [70]	Systematic review and meta-analysis	13 cross-sectional, 1 prospective cohort	Excess body weight and obesity	UPF consumption was associated with increased risk of overweight and obesity
Chen et al. [71]	Systematic review	8 cross-sectional, 12 prospective cohort	Any health outcome	UPF consumption was associated with increased risk of all-cause mortality, overall cardiovascular diseases, coronary heart diseases, cerebrovascular diseases, hypertension, metabolic syndrome, overweight and obesity, depression, irritable bowel syndrome, overall cancer, postmenopausal breast cancer, gestational obesity, adolescent asthma and wheezing, and frailty. No association with cardiovascular disease mortality, prostate and colorectal cancers, gestational diabetes mellitus, or gestational overweight
Costa et al. [106]	Systematic Review	5 interventions, 6 cross-sectional, and 15 prospective cohorts	Body fat (during childhood and adolescence)	Consumption of UPF was positively associated with body fat during childhood and adolescence
de Miranda et al. [107]	Narrative review	1 randomized controlled trial, 15 prospective cohort, 12 cross-sectional, 1 prospective and cross-sectional, and 1 ecological	Metabolic health	Consumption of UPF was positively associated with metabolic syndrome, body weight change and obesity indicators, blood pressure and hypertension, glucose profile, insulin resistance and type 2 diabetes, other metabolic risks and cardiovascular diseases and mortality
Elizabeth et al. [55]	Narrative review	1 randomized controlled trial, 1 case-control, 19 prospective cohort, 19 cross-sectional, 3 ecological	All health outcomes	Consumption of UPF was positively associated with overweight, obesity and cardio-metabolic risks; cancer, type 2 diabetes and cardiovascular diseases; irritable bowel syndrome, depression and frailty conditions; and all-cause mortality in adults. Among children and adolescents, UPF consumption was associated with included cardio-metabolic risks and asthma

Figura 3: Highlight delle occorrenze del termine «cardiovascular» nel full-text del risultato 2

Questo risultato conferma l'efficacia dell'indicizzazione **full-text**: il sistema ha saputo inferire la rilevanza tematica correlando le «comorbidità» citate nel titolo con il rischio cardiaco presente nel testo (in ambito medico, la comorbidità n.1 dell'obesità è proprio il rischio cardiovascolare), evitando un falso negativo che si sarebbe verificato con una ricerca limitata ai soli metadati.

Granularità nella Ricerca su Tabelle

La ricerca sull'indice `tables` ha evidenziato il vantaggio dell'architettura a indici separati. A differenza dei motori tradizionali dove le tabelle sono «appiattite» nel testo, il sistema ha isolato puntualmente i dati statistici richiesti.

Query: 'confidence interval' | Indice: tables | Tipo: PHRASE

#1 [Score: 31.34] ID: PMC11462998

DIDASCALIA: Hazard ratio (95% confidence interval) for type 2 diabetes incidence according to ultra-process food consumption...

- Rilevante? (y/n): y

#2 [Score: 30.75] ID: PMC9414627

DIDASCALIA: Adjusted means (95% confidence interval), percentage difference, and p for a linear trend across quintiles of raw food consumption....

- Rilevante? (y/n): y
 #3 [Score: 30.75] ID: PMC9414627
 DIDASCALIA: Adjusted means (95% confidence interval), percentage difference, and p for a linear trend across quintiles of boiled food consumption....
 - Rilevante? (y/n): y
 #4 [Score: 30.75] ID: PMC9414627
 DIDASCALIA: Adjusted means (95% confidence interval), percentage difference, and p for a linear trend across quintiles of roasted food consumption....
 - Rilevante? (y/n): y
 #5 [Score: 30.17] ID: PMC9669963
 DIDASCALIA: Crude and predicted mean and 95% confidence interval of daily per capita purchase of minimally processed and ultra-processed beverages....
 - Rilevante? (y/n): y

È interessante notare come i risultati 2, 3 e 4 abbiano lo stesso score (30.75). Ciò accade perché appartengono allo stesso paper ma rappresentano tabelle diverse (es. consumo di cibo «crudo» vs «bollito»). Questo comportamento non è un difetto, ma una conferma della **granularità** dell'indice: l'utente può atterrare direttamente sulla specifica tabella di interesse invece che genericamente sull'articolo.

Ricerca Visuale e Ranking

Per quanto riguarda le figure, il fattore determinante è stato il peso assegnato al campo `caption` (³). Poiché le didascalie descrivono esplicitamente il contenuto visivo (es. «**Figure 4: Learning curve...**»), il sistema è riuscito a isolare i grafici di performance dal resto del testo narrativo.

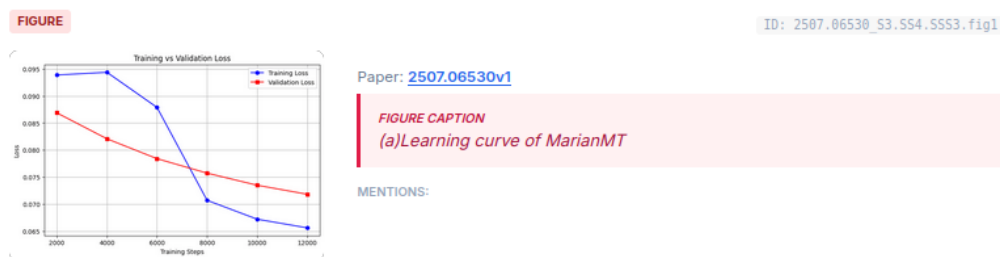


Figura 4: Primo risultato per la query «learning curve». Il sistema identifica correttamente il grafico di training.

2.3. Conclusioni

La sperimentazione ha validato l'efficacia dell'architettura multi-indice proposta. L'analisi incrociata dei dati evidenzia come il costo computazionale rilevato (latenza maggiore per query complesse) sia pienamente giustificato dall'elevata qualità del recupero informativo.

Il punto di forza del sistema risiede nella **granularità**: la separazione tra **Articles**, **Tables** e **Figures** ha permesso di raggiungere una precisione del 100% su query tecniche, recuperando dati e grafici che i motori tradizionali avrebbero ignorato. La configurazione dei pesi (**boosting**) si è dimostrata efficace nel garantire un ranking coerente con la rilevanza semantica.